DATA ENGINEERING PREPARATION

DAY 8 EXERCISES

 # LINUX PRACTICE

Linux Task 1 - Inspection revision

1. Count total lines including header.

2. Count data rows only (exclude header).

3. Display only the header line.

4. Display last 3 rows.

5. Search for all patients from cities

   that contain "a" (case-insensitive).

   Use grep -i.

6. Count how many patients have blood type O+.

1. Count total lines including header.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ cat patients.csv | wc -l
9
```

2. Count data rows only (exclude header).

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ tail -n +2 patients.csv | column -
t -s,
1  Raj Kumar        34  O+   Delhi    500
2  Sneha Patel      39  A+   Mumbai   200
3  Tom Williams     46  B-   Chicago  750
4  Fatima Al-Sayed  29  AB+  Dubai    600
5  Liu Yang         24  O-   Beijing  300
6  Ana Gonzalez     56  A-   Madrid   900
7  James Okafor     32  B+   Lagos    400
8  Emily Carter     19  O+   Toronto  150
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$
```

3. Display only the header line.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ head -n 1 patients.csv | column -t
 -s,
patient_id  name  age  blood_type  city  fee
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$
```

4. Display last 3 rows.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ tail -n 3 patients.csv | column -t
 -s,
6  Ana Gonzalez  56  A-  Madrid   900
7  James Okafor  32  B+  Lagos    400
8  Emily Carter  19  O+  Toronto  150
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$
```

5. Search for all patients from cities

that contain "a" (case-insensitive).

Use grep -i.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ grep -i "a" patients.csv
patient_id,name,age,blood_type,city,fee
1,Raj Kumar,34,O+,Delhi,500
2,Sneha Patel,39,A+,Mumbai,200
3,Tom Williams,46,B-,Chicago,750
4,Fatima Al-Sayed,29,AB+,Dubai,600
5,Liu Yang,24,O-,Beijing,300
6,Ana Gonzalez,56,A-,Madrid,900
7,James Okafor,32,B+,Lagos,400
8,Emily Carter,19,O+,Toronto,150
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$
```

6. Count how many patients have blood type O+.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ grep -c "O+" patients.csv
2
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ grep -i "o+" patients.csv
1,Raj Kumar,34,O+,Delhi,500
8,Emily Carter,19,O+,Toronto,150
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$
```

Linux Task 2 - Text processing revision

1. Extract only the name column.

2. Extract name and fee columns together.

3. List all unique cities in the file.

Use cut + sort + uniq.

4. Count patients per city.

Use cut + sort + uniq -c + sort -rn.

5. Find the patient with the highest fee

without using Python.

Hint: sort -t',' -k6 -rn | head -2

(head -2 to skip the header that sort

may push down)

1. Extract only the name column.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ cut -d',' -f2 patients.csv
name
Raj Kumar
Sneha Patel
Tom Williams
Fatima Al-Sayed
Liu Yang
Ana Gonzalez
James Okafor
Emily Carter
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$
```

2. Extract name and fee columns together.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ cut -d',' -f2,6 patients.csv
name,fee
Raj Kumar,500
Sneha Patel,200
Tom Williams,750
Fatima Al-Sayed,600
Liu Yang,300
Ana Gonzalez,900
James Okafor,400
Emily Carter,150
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$
```

3. List all unique cities in the file.

Use cut + sort + uniq.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ cut -d',' -f5 patients.csv | sort
| uniq
Beijing
Chicago
Delhi
Dubai
Lagos
Madrid
Mumbai
Toronto
city
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$
```

4. Count patients per city.

Use cut + sort + uniq -c + sort -rn.

```
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ cut -d',' -f5 patients.csv | sort
| uniq -c | sort -rn
      1 city
      1 Toronto
      1 Mumbai
      1 Madrid
      1 Lagos
      1 Dubai
      1 Delhi
      1 Chicago
      1 Beijing
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ S
```

5. Find the patient with the highest fee

without using Python.

Hint: sort -t',' -k6 -rn | head -2

(head -2 to skip the header that sort

may push down)

```
      1 Beijing
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$ sort -t',' -k6 -rn patients.csv |
head -2
6,Ana Gonzalez,56,A-,Madrid,900
3,Tom Williams,46,B-,Chicago,750
prashant@DESKTOP-B9KE2N7:/mnt/d/kode-x notes/python/python_exercise's/day6_project/order_pipeline$
```

 # SQL PRACTICE

SQL Exercise 1 - WHERE + NULL + ORDER

1. Display all employees where salary is NULL.

   Replace NULL salary with 0 in the output.

   Use NVL. Do not update the table.

2. Display employees where department is NULL

   OR salary is NULL.

3. Display active employees (is_active = 1)

   from Engineering or Finance,

   with salary above 70000,

   ordered by salary descending.

4. Display employees hired between

   2019-01-01 and 2021-12-31,

   who are still active,

   ordered by hire_date ascending.

5. Display top 5 highest paid active employees.

   Show name, department, salary.

   Use FETCH FIRST 5 ROWS ONLY.

1. Display all employees where salary is NULL.

   Replace NULL salary with 0 in the output.

   Use NVL. Do not update the table.

```
3   --    Use NVL. Do not update the table.
4
5   SELECT EMP_ID, NAME, SALARY FROM EMPLOYEES WHERE SALARY IS NULL;
6   SELECT NAME, NVL(SALARY, 0) FROM EMPLOYEES;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.001 seconds

	NAME	NVL(SALARY,0)
7	Henry Davis	71000
8	Ian Wilson	0
9	Jane Taylor	72000
10	Kyle Adams	0

2. Display employees where department is NULL

OR salary is NULL.

```
-- 2. Display employees where department is NULL
--    OR salary is NULL.

SELECT * FROM EMPLOYEES WHERE DEPARTMENT IS NULL OR SALARY IS NULL;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.001 seconds

	EMP_ID	NAME	SALARY	DEPARTMENT	HIRE_DATE	IS_ACTIVE
1	9	Ian Wilson	*(null)*	Marketing	1/15/2023, 12:00:0(	1
2	10	Jane Taylor	72000	*(null)*	3/15/2023, 12:00:0(	1
3	11	Kyle Adams	*(null)*	*(null)*	8/20/2022, 12:00:0(	0

3. Display active employees (is_active = 1)

from Engineering or Finance,

with salary above 70000,

ordered by salary descending.

SQL Worksheet

```
-- 3. Display active employees (is_active = 1)
--    from Engineering or Finance,
--    with salary above 70000,
--    ordered by salary descending.

SELECT * FROM EMPLOYEES
WHERE IS_ACTIVE = 1
    AND DEPARTMENT IN ('Engineering', 'Finance')
    AND SALARY > 70000
ORDER BY SALARY DESC;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.005 seconds

	EMP_ID	NAME	SALARY	DEPARTMENT	HIRE_DATE	IS_ACTIVE
1	12	Linda Chen	95000	Engineering	6/10/2016, 12:00:0(	1
2	3	Carol Williams	92000	Engineering	11/20/2018, 12:00:(	1
3	6	Frank Lee	88000	Engineering	5/22/2017, 12:00:0(	1
4	1	Alice Johnson	85000	Engineering	3/15/2020, 12:00:0(	1

4. Display employees hired between

2019-01-01 and 2021-12-31,

who are still active,

ordered by hire_date ascending.

```
-- 4. Display employees hired between
--    2019-01-01 and 2021-12-31,
--    who are still active,
--    ordered by hire_date ascending.

SELECT NAME, HIRE_DATE FROM EMPLOYEES WHERE HIRE_DATE BETWEEN DATE '2019-01-01' AND DATE '2021-12-31';
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.003 seconds

	NAME	HIRE_DATE
1	Alice Johnson	3/15/2020, 12:00:00 AM
2	Bob Smith	7/1/2019, 12:00:00 AM
3	David Brown	1/10/2021, 12:00:00 AM
4	Henry Davis	12/1/2019, 12:00:00 AM

5. Display top 5 highest paid active employees.

Show name, department, salary.

Use FETCH FIRST 5 ROWS ONLY.

```
-- 5. Display top 5 highest paid active employees.
--    Show name, department, salary.
--    Use FETCH FIRST 5 ROWS ONLY.

SELECT NAME, DEPARTMENT, NVL(SALARY, 0) FROM EMPLOYEES
WHERE IS_ACTIVE = 1
ORDER BY NVL(SALARY, 0) DESC
FETCH FIRST 5 ROW ONLY;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.004 seconds

	NAME	DEPARTMENT	NVL(SALARY,0)
1	Linda Chen	Engineering	95000
2	Carol Williams	Engineering	92000
3	Frank Lee	Engineering	88000
4	Alice Johnson	Engineering	85000

SQL Exercise 2 - Aggregation + GROUP BY revision

1. Find total salary expenditure per department.
   Exclude NULL departments.
   Order by total descending.
2. Find average salary per department.
   Show only departments where average
   salary exceeds 70000.
   Use HAVING.
3. Count active vs inactive employees.
   Group by is_active.
   Show 1 → Active, 0 → Inactive as labels.
   Hint: Use CASE inside SELECT.
4. Find the department with the highest
   total salary bill.
   Use a subquery or FETCH FIRST 1 ROW ONLY.
5. For the orders table:
   Find total revenue and order count
   per order status.
   Exclude cancelled orders from the total.
   Order by total revenue descending.

1. Find total salary expenditure per department.

Exclude NULL departments.

Order by total descending.

```sql
-- 1. Find total salary expenditure per department.
--    Exclude NULL departments.
--    Order by total descending.

SELECT DEPARTMENT,SUM(SALARY) AS TOTAL_SALARY
FROM EMPLOYEES
WHERE DEPARTMENT IS NOT NULL
GROUP BY DEPARTMENT
ORDER BY TOTAL_SALARY DESC;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.003 seconds

	DEPARTMENT	TOTAL_SALARY
1	Engineering	360000
2	Finance	219000
3	HR	114000
4	Marketing	62000

2. Find average salary per department.

Show only departments where average

salary exceeds 70000.

Use HAVING.

```sql
-- 2. Find average salary per department.
--    Show only departments where average
--    salary exceeds 70000.
--    Use HAVING

SELECT DEPARTMENT,
    AVG(SALARY) AS AVG_SALARY
    FROM EMPLOYEES
GROUP BY DEPARTMENT
HAVING AVG(SALARY) > 70000;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.003 seconds

	DEPARTMENT	AVG_SALARY
1	Engineering	90000
2	*(null)*	72000
3	Finance	73000

4. Find the department with the highest

total salary bill.

Use a subquery or FETCH FIRST 1 ROW ONLY.

```
-- 4. Find the department with the highest
--    total salary bill.
--    Use a subquery or FETCH FIRST 1 ROW ONLY.

SELECT DEPARTMENT, SUM(SALARY) AS TOTAL_SALARY
FROM EMPLOYEES
    GROUP BY DEPARTMENT
    ORDER BY TOTAL_SALARY DESC
    FETCH FIRST 1 ROW ONLY;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.003 seconds

	DEPARTMENT	TOTAL_SALARY
1	Engineering	360000

5. For the orders table: Find total revenue and order count

per order status.

Exclude cancelled orders from the total.

Order by total revenue descending.

```
SELECT * FROM ORDERS;
SELECT STATUS,
    SUM(QUANTITY * AMOUNT) AS TOTAL_REVENUE,
    COUNT(*) AS ORDER_COUNT
FROM ORDERS
    WHERE STATUS <> 'Cancelled'
    GROUP BY STATUS
    ORDER BY TOTAL_REVENUE DESC;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.006 seconds

	STATUS	TOTAL_REVENUE	ORDER_COUNT
1	Shipped	6979.79	3
2	Delivered	3948.83	7
3	Pending	1039.94	3

SQL Exercise 3 - Multi-concept challenge

Write a single query for each:

1. From employees:

   Show department, headcount, avg salary,

   min salary, max salary.

   Only include active employees.

   Only show departments with more than

   1 active employee.

   Order by avg salary descending.

```
7   --    Order by avg salary descending.
8   SELECT DEPARTMENT,
9       COUNT(*) AS HEADCOUNT,
10      AVG(SALARY) AS AVG_SALARY,
11      MIN(SALARY) AS MINIMUM_SALARY,
12      MAX(SALARY) AS MAXIMUM_SALARY
13  FROM EMPLOYEES
14      WHERE IS_ACTIVE = 1
15      GROUP BY DEPARTMENT
16      HAVING COUNT(*) > 1
17      ORDER BY AVG_SALARY DESC;
18
19
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.003 seconds

	DEPARTMENT	HEADCOUNT	AVG_SALARY	MINIMUM_SALARY	MAXIMUM_SALARY
1	Engineering	4	90000	85000	95000
2	Finance	3	73000	67000	81000
3	Marketing	2	62000	62000	62000
4	HR	2	57000	55000	59000

2. From orders and products:

For each product_id that appears in orders,

show product_id, total quantity ordered,

total revenue (quantity x amount per row),

and number of distinct customers.

Order by total revenue descending.

```
6   --    Order by total revenue descending.
7   SELECT * FROM ORDERS;
8   SELECT PRODUCT_ID,
9       SUM(QUANTITY) AS TOTAL_QUANTITY,
10      SUM(QUANTITY * AMOUNT) AS TOTAL_REVENUE,
11      COUNT(DISTINCT CUSTOMER_ID) AS DISTINCT_CUSTOMERS
12  FROM ORDERS
13  GROUP BY PRODUCT_ID
14  ORDER BY TOTAL_REVENUE DESC;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.006 seconds

	PRODUCT_ID	TOTAL_QUANTITY	TOTAL_REVENUE	DISTINCT_CUSTOMERS
1	101	4	7799.94	3
2	108	2	899.98	2
3	110	10	899	1
4	104	1	599.99	1

 # PYTHON PRACTICE

 # Exercise 1 - Function revision with real data logic

Write the following functions:

1. get_active_employees(employees)

   Returns list of only active employees.

2. average_salary_by_dept(employees)

   Returns dict: {dept: avg_salary}

   Round to 2 decimal places.

3. top_earner(employees)

   Returns the name and salary of the

   highest paid employee.

4. salary_band(employees)

   Returns a new list of dicts with an

   extra key "band" added to each employee:

   salary >= 80000 → "Senior"

   salary >= 60000 → "Mid"

   below 60000 → "Junior"

5. dept_headcount(employees)

   Returns dict: {dept: count}

   Only count active employees.

```python
# Exercise 1 - Function revision with real data logic
employees = [
    {"name": "Alice",  "dept": "Engineering", "salary": 85000, "active": True},
    {"name": "Bob",    "dept": "Marketing",   "salary": 62000, "active": True},
    {"name": "Carol",  "dept": "Engineering", "salary": 92000, "active": True},
    {"name": "David",  "dept": "HR",          "salary": 55000, "active": False},
    {"name": "Eva",    "dept": "Marketing",   "salary": 78000, "active": False},
    {"name": "Frank",  "dept": "Engineering", "salary": 88000, "active": True},
    {"name": "Grace",  "dept": "Finance",     "salary": 67000, "active": True},
]
# 1. get_active_employees(employees), Returns list of only active employees.
def get_active_employees(employees):
    return [i for i in employees if i['active']]

# 2. average_salary_by_dept(employees)
def average_salary_by_dept(employees):
    department_data = {}

    for i in employees:
        dept = i['dept']
        salary = i['salary']

        if dept not in department_data:
            department_data[dept] = {"total": 0, "count": 0}

        department_data[dept]['total'] += salary
        department_data[dept]['count'] += 1
```

```
    return {dept: round(data['total'] / data['count'], 2) for dept, data in department_data.items()}

# 3. top_earner(employees)
def top_earner(employees):
    top = max(employees, key= lambda x: x['salary'])
    return top['name'], top['salary']

# 4. salary_band(employees)
def salary_band(employees):
    add_band = []

    for i in employees:
        new_employee = i.copy()

        if i['salary'] >= 80000:
            new_employee["band"] = "Senior"
        elif i['salary'] >= 60000:
            new_employee["band"] = "Mid"
        else:
            new_employee["band"] = "Junior"
        add_band.append(new_employee)

    return add_band
```

```
# 5. dept_headcount(employees)
def dept_headcount(employees):
    counts_dict = {}

    for i in employees:
        if i['active']:
            dept = i['dept']
            counts_dict[dept] = counts_dict.get(dept, 0) + 1

    return counts_dict

active_employee = get_active_employees(employees)
avg_salary = average_salary_by_dept(employees)
top_name, top_salary = top_earner(employees)
banded_column = salary_band(employees)
no_of_department = dept_headcount(employees)
```

```
print("-------------------------------")
print("Active Employees:")
print("-------------------------------")
for i in active_employee:
    print(f"{i['name']} - {i['dept']} - {i['salary']}")

print("\nAverage Salary by Department:")
for dept, avg in avg_salary.items():
    print(f"{dept}: {avg}")

print(f"\nTop Earner: {top_name} with salary: {top_salary}")

print("\nEmployees with Salary Band:")
for i in banded_column:
    print(f"{i['name']} - {i['salary']} - {i['band']}")

print("\nDepartment Headcount (Avtive Employee Only):")
for dept, count in no_of_department.items():
    print(f"{dept}: {count}")
```

Output :-

```
...   ------------------------------
      Active Employees:
      ------------------------------
      Alice - Engineering - 85000
      Bob - Marketing - 62000
      Carol - Engineering - 92000
      Frank - Engineering - 88000
      Grace - Finance - 67000

      Average Salary by Department:
      Engineering: 88333.33
      Marketing: 70000.0
      HR: 55000.0
      Finance: 67000.0

      Top Earner: Carol with salary: 92000

      Employees with Salary Band:
      Alice - 85000 - Senior
      Bob - 62000 - Mid
      Carol - 92000 - Senior
      David - 55000 - Junior
      Eva - 78000 - Mid
      Frank - 88000 - Senior
      Grace - 67000 - Mid
      ...
      Department Headcount (Avtive Employee Only):
      Engineering: 3
      Marketing: 1
      Finance: 1
```

 # Exercise 2 - Exception Handling

Write a function called safe_divide(a, b) that:

- Returns a divided by b

- Handles ZeroDivisionError gracefully

- Handles TypeError if non-numbers are passed

- Logs a message for each error type

- Returns None on error instead of crashing

Then write a function called process_salaries(data)

that takes a list like:

[85000, 0, "N/A", 62000, None, 78000]

For each value, try dividing 1200000 by it

(simulating yearly-to-monthly).

Use safe_divide and print either the result

or the error reason.

```python
# Exercise 2 - Exception Handling
def safe_divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        print(f"Error: Cannot divided {a} by zero")
        return None
    except TypeError:
        print(f"Error: Invalid type for division - a={a}, b={b}")
        return None

def process_salaries(data):
    for value in data:
        result = safe_divide(1200000, value)

        if result is not None:
            print(f"Monthly value for {value}: {result}")
        else:
            print(f"Skipping invalid input: {value}")

data = [85000, 0, "N/A", 62000, None, 78000]
process_salaries(data)
```

```
Monthly value for 85000: 14.117647058823529
Error: Cannot divided 1200000 by zero
Skipping invalid input: 0
Error: Invalid type for division - a=1200000, b=N/A
Skipping invalid input: N/A
Monthly value for 62000: 19.35483870967742
Error: Invalid type for division - a=1200000, b=None
Skipping invalid input: None
Monthly value for 78000: 15.384615384615385
```

Exercise 3 - File Handling revision: JSON

You are given this JSON string (paste it

into your code as a variable or save as

employees.json):

Write code to:

1. Parse the JSON using the json module.

2. Print the company name.

3. Print all employee names and salaries.

4. Find and print the highest paid employee.

5. Add a new employee to the list:

   id 6, name "Henry", dept "Finance",

   salary 71000.

6. Write the updated data back to employees.json

   with indent=2.

7. Read and verify the file was written correctly.

```
employee_data = {
  "company": "KloudOcean",
  "employees": [
    {"id": 1, "name": "Alice",  "dept": "Engineering", "salary": 85000},
    {"id": 2, "name": "Bob",    "dept": "Marketing",   "salary": 62000},
    {"id": 3, "name": "Carol",  "dept": "Engineering", "salary": 92000},
    {"id": 4, "name": "David",  "dept": "HR",          "salary": 55000},
    {"id": 5, "name": "Grace",  "dept": "Finance",     "salary": 67000}
  ]
}
import json
with open("employees.json", "w") as file:
    json.dump(employee_data, file, indent=2)
print("-------------------------------------------")
# 2. Print the company name.
print(f"Company name: \'{employee_data['company']}\'")

print("-------------------------------------------")
# 3. Print all employee names and salaries.
new_list = employee_data['employees']
print(type(new_list))
for i in new_list:
    print(f"Name: {i['name']}, Salary: {i['salary']}")
```

```
print("-----------------------------------------------")
# 4. Find and print the highest paid employee.
new_list = employee_data['employees']
top = max(new_list, key= lambda x: x['salary'])
print(f"Highest paid employee: \nName: {top['name']} \nSalary: {top['salary']}")

print("-----------------------------------------------")
# 5. Add a new employee to the list:
#    id 6, name "Henry", dept "Finance",
#    salary 71000.
new_emp = {"id": 6, "name": "Henry", "dept": "Finance", "salary": 71000}
employee_data["employees"].append(new_emp)
print(f"Add new employee after: {employee_data}")

# 6. Write the updated data back to employees.json
#    with indent=2.
with open("employees.json", "w") as file:
    json.dump(employee_data, file, indent=2)

print("-----------------------------------------------")
# 7. Read and verify the file was written correctly.
with open("employees.json", "r") as file:
    read_data = json.load(file)
    print("\nVerification:")
    for row in read_data['employees']:
        print(f"{row['name']} - {row['salary']}")
```

Output :-

```
-----------------------------------------------
Company name: 'KloudOcean'
-----------------------------------------------
<class 'list'>
Name: Alice, Salary: 85000
Name: Bob, Salary: 62000
Name: Carol, Salary: 92000
Name: David, Salary: 55000
Name: Grace, Salary: 67000
-----------------------------------------------
Highest paid employee:
Name: Carol
Salary: 92000
-----------------------------------------------
Add new employee after: {'company': 'KloudOcean', 'employees': [{'id': 1, 'name': 'Alice', 'dept': 'Engineering'
-----------------------------------------------

Verification:
Alice - 85000
Bob - 62000
Carol - 92000
David - 55000
Grace - 67000
Henry - 71000
```

 # Exercise 4 - Pandas: First Steps with CSV

Create a file called patients.csv with this data:

Now write a Python script using pandas:

1. Read the CSV into a DataFrame.

import pandas as pd

df = pd.read_csv("patients.csv")

```
import pandas as pd

data = """patient_id,name,age,blood_type,city,fee
1,Raj Kumar,34,O+,Delhi,500
2,Sneha Patel,39,A+,Mumbai,200
3,Tom Williams,46,B-,Chicago,750
4,Fatima Al-Sayed,29,AB+,Dubai,600
5,Liu Yang,24,O-,Beijing,300
6,Ana Gonzalez,56,A-,Madrid,900
7,James Okafor,32,B+,Lagos,400
8,Emily Carter,19,O+,Toronto,150
"""
with open("patients.csv", "w", newline="") as file:
    file.write(data)

# 1. Read the CSV into a DataFrame.
df = pd.read_csv("patients.csv")
print(df)
```

Output :-

```
   patient_id             name  age blood_type     city  fee
0           1        Raj Kumar   34         O+    Delhi  500
1           2      Sneha Patel   39         A+   Mumbai  200
2           3     Tom Williams   46         B-  Chicago  750
3           4  Fatima Al-Sayed   29        AB+    Dubai  600
4           5         Liu Yang   24         O-  Beijing  300
5           6     Ana Gonzalez   56         A-   Madrid  900
6           7     James Okafor   32         B+    Lagos  400
7           8     Emily Carter   19         O+  Toronto  150
```

 # Exercise 5 - Pandas: DataFrame Operations

Using the same patients.csv:

1. Add a new column called fee_category:

   fee >= 700 → "High"

   fee >= 400 → "Medium"

   below 400 → "Low"

2. Add a new column called age_group:

   age < 30 → "Young"

   age < 50 → "Adult"

   age >= 50 → "Senior"

3. Find the average fee per city. [Use groupby()].

4. Count patients per blood_type. [Use value_counts()].

5. Find the city with the highest average fee.

6. Export the updated DataFrame to a new file:

   patients_processed.csv

   Do not include the index.

1. Add a new column called fee_category:

   fee >= 700 → "High"

   fee >= 400 → "Medium"

   below 400 → "Low"

```
def categorize_fee(fee):
    if fee >= 700:
        return "High"
    elif fee >= 400:
        return "Medium"
    else:
        return "Low"
df["fee_category"] = df["fee"].apply(categorize_fee)
```

2. Add a new column called age_group:

age < 30 → "Young"

age < 50 → "Adult"

age >= 50 → "Senior"

```
# 2. Add a new column called age_group:
#    age < 30   → "Young"
#    age < 50   → "Adult"
#    age >= 50  → "Senior"

def age_g(age):
    if age < 30:
        return "Young"
    elif age < 50:
        return "Adult"
    else:
        return "Senior"
df["age_group"] = df["age"].apply(age_g)
```

3. Find the average fee per city. [Use groupby()].

```
# 3. Find the average fee per city.
#    Use groupby().
average_fee = df.groupby("city")["fee"].mean()
print(f"Average per city is:\n{average_fee}")

# Explanation:
    # groupby("city") → groups rows by city
    # ["fee"].mean() → computes average fee per group
```

✓ 0.0s

```
Average per city is:
city
Beijing    300.0
Chicago    750.0
Delhi      500.0
Dubai      600.0
Lagos      400.0
Madrid     900.0
Mumbai     200.0
Toronto    150.0
Name: fee, dtype: float64
```

4. Count patients per blood_type. [Use value_counts()].

```
# 4. Count patients per blood_type.
#    Use value_counts().
blood_types = df["blood_type"].value_counts()
print(f"Count: {blood_types}")

# Explanation:
    # Counts occurrences of each blood group
```

✓ 0.0s

```
Count: blood_type
O+     2
A+     1
B-     1
AB+    1
O-     1
A-     1
B+     1
Name: count, dtype: int64
```

5. Find the city with the highest average fee.

```
# 5. Find the city with the highest average fee.
top_patient = average_fee.idxmax()
print(f"Highest average fee patients city is: {top_patient}")

# Explanation:
    # idxmax() returns the index (city) with max value
```

✓ 0.0s

```
Highest average fee patients city is: Madrid
```

6. Export the updated DataFrame to a new file:

patients_processed.csv

Do not include the index.

```
# 6. Export the updated DataFrame to a new file:
#    patients_processed.csv
#    Do not include the index.
df.to_csv("patients_processed.csv", index=False)
print(df)
```

✓ 0.0s

```
   patient_id             name  age blood_type     city  fee fee_category  \
0           1        Raj Kumar   34         O+    Delhi  500       Medium
1           2      Sneha Patel   39         A+   Mumbai  200          Low
2           3     Tom Williams   46         B-  Chicago  750         High
3           4  Fatima Al-Sayed   29        AB+    Dubai  600       Medium
4           5         Liu Yang   24         O-  Beijing  300          Low
5           6     Ana Gonzalez   56         A-   Madrid  900         High
6           7     James Okafor   32         B+    Lagos  400       Medium
7           8     Emily Carter   19         O+  Toronto  150          Low

  age_group
0     Adult
1     Adult
2     Adult
3     Young
4     Young
5    Senior
6     Adult
7     Young
```